QUAX Streaming Pipeline

Architecture and Implementation Notes

MAPD-B — Methods and Applications of Physics Data
Project 6 — Distributed Streaming Pipeline for the QUAX Axion Detector

Esteban Fernando Cárdenas Andino
Eylul Çagla Ersoz
Melissa Daniela de Almeida Nespeque
Vinícius de Souza Martins

1. Introduction

This document describes the cooperating components implemented for Project 6: a producer that emulates the QAX detector front-end, a distributed processing engine that reconstructs and analyzes the streamed data using Apache Kafka and Dask, and a live Streamlit dashboard serving as the consumer. The goal of the pipeline is to take raw in-phase/quadrature (I/Q) samples, recover the power spectrum of each acquisition via the Fast Fourier Transform (FFT), and continuously merge the results into a single, ever-improving global running average and variance. This statistical spectrum is the core quantity of interest in the search for a relic axion signal inside the QUAX microwave cavity. The notes below are organized by file, detailing script responsibilities, core distributed computing concepts, and the mathematical foundations behind the signal-processing and online statistical steps.

2. System Architecture

The pipeline follows a decoupled streaming architecture distributed across a 5-node bare-metal cluster. The Producer fetches raw data from S3 and publishes binary frames onto a dedicated Kafka topic (topic_stream). The Dask-powered processing engine consumes this topic, executes parallelized FFT analysis across distributed compute workers, and updates the global spectrum state using Welford's online algorithm. The updated results are republished onto a second Kafka topic (topic_results), which is continuously polled by the Streamlit consumer application for live visualization.

[VM1] Producer → [VM2] Kafka (`topic_stream`) → [VM3, VM4, VM5] **Dask Processing engine (Dask)** → [VM2] Kafka (`topic_results`) → [VM1] **Streamlit Dashboard**

Component	Machine	Script	Role
Producer	VM1 (10.67.22.42)	producer.py	Downloads S3 I/Q binaries, chunks data into 65k scans, applies rate-pacing, and round-robin streams frames to Kafka with an end-of-stream signal.
Visualization	VM1	dashboard.py	Streamlit app that polls Kafka results, displays peak/floor power metrics, and plots the real-time Power Spectral Density chart in MHz
Consumer	VM2 (10.67.22.134)	Apache Kafka	Deployed in KRaft mode as a high-throughput queue buffering raw input frames and compiled spectral results.

Component	Machine	Script	Role
Processing engine	VM3 (10.67.22.72)	processing.py	Consumes Kafka scans, reassembles file matrices, manages Dask graph execution, merges spectrums via Welford's algorithm, and exports metrics.
Compute Muscle	VM3, VM4 (10.67.22.202), VM5 (10.67.22.185)	Dask Workers	Distributed computational nodes that load matrices into RAM over TCP to execute parallel, vectorized SciPy FFTs.

3. Producer — producer.py

The producer simulates the data acquisition front end of the detector. For each acquisition index from 0 to 30, it downloads two raw binary files from S3 containing the same physical capture split into its quadrature components: the in-phase samples (duck_i_NNNNN.dat) and the quadrature samples (duck_q_NNNNN.dat). Both arrays are dynamically truncated to a matching minimum length and sliced into fixed-size windows of 65536 samples per channel. Each window is called a scan. One scan is streamed per Kafka message, using a dynamic size-based pacing algorithm to throttle and tune the incoming stream to a specific, parameter-driven megabytes-per-second rate. Adjusting this tuning variable across different runs allows the system to systematically evaluate the cluster performance boundaries under controlled levels of stress. This active rate tuning is crucial to prevent downstream buffer congestion and memory saturation while uncovering the absolute limits of the processing architecture. To bypass Python Global Interpreter Lock restrictions and maximize network throughput, the pipeline relies on the confluent-kafka library. By utilizing this C-compiled librdkafka backend, the producer streams the tuned binary structures efficiently without network bottlenecks.

3.1 Pairing of I and Q files

The same file identifier and slicing indices are used to cut both data arrays within the same loop iteration, ensuring that every Kafka message carries the paired in-phase and quadrature chunks that belong together. This is the stage where the pair of files is fused into a single logical unit. Downstream processing components receive complete I/Q packages and never need to handle or synchronize the in-phase and quadrature files separately.

3.2 Binary message framing

Instead of serializing the full high-throughput float arrays as text or JSON, the producer builds a compact binary frame inline inside the loop to minimize payload size. The frame consists of a 4-byte big-endian unsigned integer indicating the length of the header, followed by a UTF-8 encoded JSON metadata object, and finishes with the raw little-endian single-precision float bytes of the in-phase chunk and quadrature chunk. The metadata header contains the file identifier, the scan index, the total scan count, the sample length of 65536, and the precise epoch timestamp recorded at transmission time to track end-to-end

latency. This framing layout allows the downstream consumer to reconstruct the NumPy arrays from the raw bytes using memory views at minimal computational cost.

4. Processing Engine — processing.py

The processing engine acts as a real-time cluster orchestrator. It manages a persistent Kafka consumer to capture raw data streams, handles distributed task delegation across remote Dask compute nodes, performs high-resolution latency tracking, and runs incremental signal updates. It relies on a non-blocking asynchronous architecture driven by automated callbacks to handle high-throughput workloads without blocking the primary ingestion thread.

4.1 Message parsing

The parsing mechanism decodes incoming byte streams by extracting the initial 4-byte big-endian length prefix to establish JSON metadata boundaries. If the decoded metadata contains an explicit end-of-stream event marker, the consumer logs the partition signature and updates the shutdown state tracking variables. For data payloads, the engine extracts the sample length parameters and utilizes memory-efficient parsing to extract the independent in-phase and quadrature arrays directly into single-precision float structures.

4.2 Immediate Stream Processing

The engine eliminates batch-level buffering and matrix stacking. Scans are processed sequentially as they arrive from the message partitions. As soon as the raw byte arrays are decoded into memory, they are submitted to the distributed Dask worker pool using asynchronous future allocations. This approach reduces buffer latency and ensures a continuous flow of data across the compute network.

4.3 Distributed Task Execution

Scientific computations run inside the memory space of the distributed cluster using persistent background daemons. Although VMs 4 and 5 do not execute custom project application scripts, they run the native Dask worker daemon process initialized during the cluster boot sequence. This background process transforms each remote machine into an active RPC server listening over TCP. When client.submit invokes the compute_fft task, the central Dask Scheduler serializes the Python function bytecode along with the raw in-phase and quadrature arrays and transmits them over the network. The remote Dask worker daemons intercept this TCP stream, load the single-precision array components directly into their local RAM, and execute the vectorized SciPy FFT calculations concurrently using their own CPU resources. Once processed, the workers serialize the raw power spectra and execution timestamps, returning them over the established network sockets back to the orchestration node.

4.4 Online Spectral Aggregation

When a Dask task finishes, a non-blocking callback routes the power spectrum into a thread-safe queue. The main loop drains this queue and updates a global running mean inline, instantly discarding raw signal states to keep memory overhead near zero.

4.5 Telemetry & Latency Profiling

The engine tracks pipeline bottlenecks across five distinct stages (end-to-end, queue wait, Dask scheduling, worker execution, and result transit). It uses Welford's online algorithm to compute exact means, standard deviations, and min/max bounds dynamically without storing historical data points.

4.6 Republishing to Kafka

To protect network and dashboard bandwidth, the orchestrator updates the UI every 10 chunks. It centralizes the spectrum via an engine-side fftshift, downsamples both frequency and power arrays by a factor of 10, and publishes the optimized JSON payload alongside peak and floor metrics to topic_results.

4.7 Graceful termination

System teardown is managed by monitoring partition assignment states. The orchestration loop continuously checks the active partition count against the registered end-of-stream signals. Once all assigned channels broadcast a termination marker, the engine waits for the active future pool to drain completely, flushes outstanding results, exports the compiled metrics to a local file, and closes all network clients cleanly.

5. Benchmarking Methodology

The benchmarking process is designed as a methodical, repeatable cycle to evaluate the cluster's behavior under two specific stress conditions: partition scalability and throughput saturation.

5.1 Configuration Assessment & Evaluation Metrics

To determine the optimal cluster configuration, we cross-reference the statistical data exported in the .pkl metric files at the end of each trial. The objective is not merely to identify the "fastest" run, but to pinpoint the system's saturation point and analyze pipeline stability under sustained pressure.

The exact criteria and metrics used to evaluate the best setup are:

- **Queue Wait Time (Backpressure):** Tracked via Welford's online statistics, this is the most critical metric. It represents the time a data packet spends waiting from its transmission until a Dask worker actively begins processing it. If this duration remains stable, the configuration successfully handles the throughput. If it snowballs over time, the system has reached saturation (data is ingested faster than the compute nodes can consume it).
- **End-to-End Latency:** The total transit and processing time from the moment a binary frame is packaged on VM1 to the moment its calculated FFT is folded into the global average on VM3. The ideal configuration delivers the lowest mean latency coupled with the lowest variance (standard deviation).
- **Task Accumulation (Consumer Lag & Pending Futures):** By analyzing the telemetry snapshots taken every 50 chunks, we evaluate the queue behavior over time. This determines whether the Kafka broker buffers and the asynchronous Dask task queues remain under control, or if they inflate continuously until the trial ends.
- **Partitioning Efficiency vs. Overhead:** By comparing the 1-partition and 3-partition architectures at a fixed 16 MB/s rate, we assess whether parallelizing the Kafka consumption genuinely reduces latency, or if the added coordination cost simply increases the Dask scheduling overhead.

5. Consumer — Live Streamlit Dashboard

The consumer serves as the live data visualizer for the streaming architecture. It is implemented as a Streamlit web application running on VM1 that subscribes to the topic_results Kafka topic on VM2. The dashboard polls this channel continuously, extracting processed spectral datasets and instantly rendering key metrics and line charts to provide real-time feedback on the experiment.

5.1 auto_offset_reset Configuration

The consumer configuration sets the auto.offset.reset parameter to 'earliest' inside a cached consumer factory function wrapper. This ensures that the visualization application reads all aggregated messages sitting on the topic from the very beginning of the trial run. This design guarantees a complete look at the session data even if the Streamlit app is opened or restarted after the processing engine has already begun publishing data.

5.2 Frequency Scale Conversion

The dashboard receives raw frequency listings from the message payload and converts them into Megahertz. This converted array serves as the direct index for a Pandas DataFrame holding the Power Spectral Density values. This layout automatically maps the physical frequency variables cleanly along the horizontal axis of the chart without needing manual sorting configurations.

5.3 Continuous Execution and Robustness

The dashboard operates inside an infinite polling loop utilizing a short timeout of 0.1 seconds to keep the application highly responsive. The consumption sequence runs inside a global try-except structure that catches data inconsistencies or parsing exceptions. This allows the script to log and bypass corrupt frames silently without crashing the dashboard, keeping the live viewport running indefinitely during high-throughput testing.

6. Kafka Topics Summary

Topic	Publisher	Subscriber	Payload
topic_stream	producer.py	processing.py	Binary frames containing a JSON metadata header and raw float32 I/Q scan arrays, plus a JSON END_OF_STREAM control marker at completion.
topic_results	processing.py	dashboard.py	JSON structures containing cumulative power spectrum arrays, frequency vectors, total chunk counters, peak power, and floor mean power metrics.

7. Conclusion

Together, the three scripts implement the architecture end to end: the producer emulates a realistic data source, the processing engine turns raw I/Q scans into a continuously refined power spectrum using Dask for parallel FFT computation and a numerically exact incremental merge across files, and the dashboard consumes that single evolving result to provide a live view suitable for monitoring the QUAX detector during a real acquisition run.

8. References

https://github.com/vinimicius/MAPD-B_Gr07/